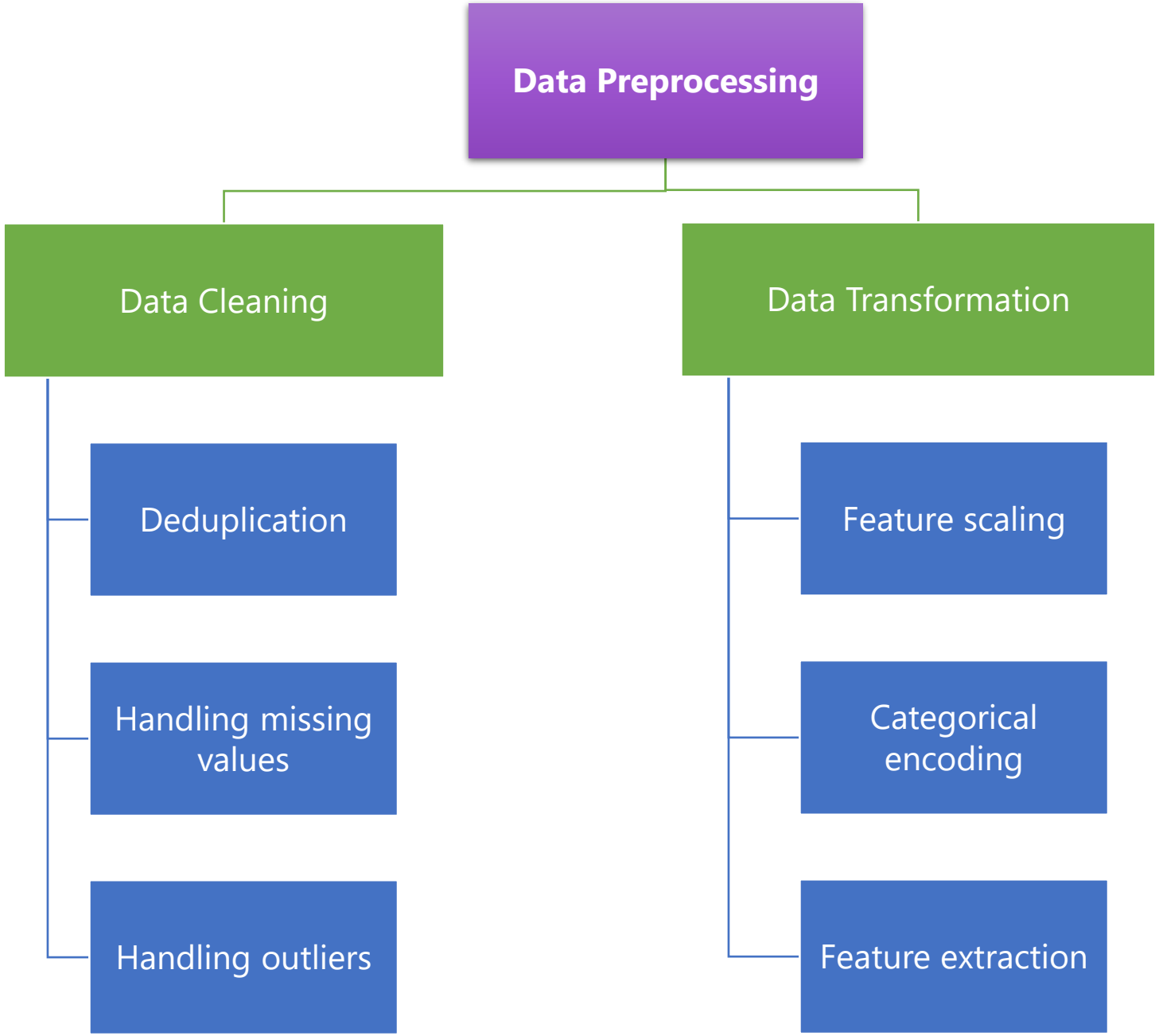
Data Preparing for Machine Learning



Outline


Data Preprocessing
Data Cleaning
Data Transformation
Deduplication
Handling missing values
Handling outliers
Feature scaling
Categorical encoding
Feature extraction

Data Deduplication





05.dp**01-deduplication**

Data Deduplication

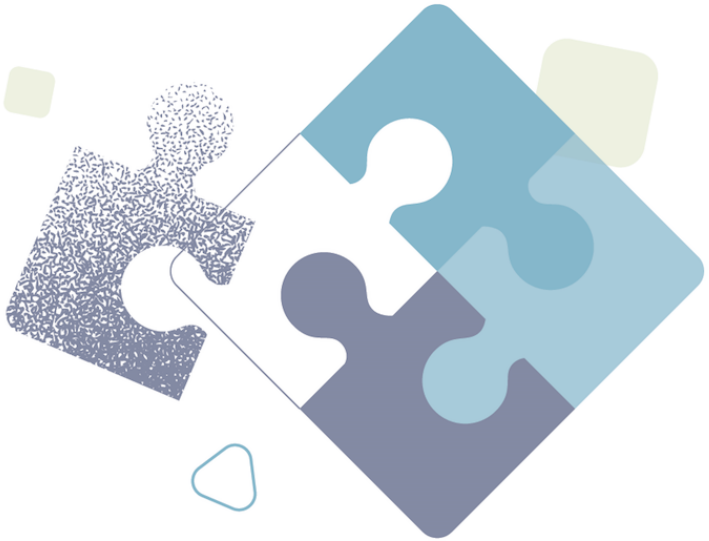
- **Duplicates:** instances with identical feature values, representing the same real-world entity or event

- Duplicate data can **bias ML models**, as repeated instances receive more influence and may distort learning outcomes. E.g.,
  - The same person submits an online form multiple times
  - Retweets share the same content as the original tweet, differing only in metadata (e.g., user, timestamp)

- Duplicate detection techniques:
  - **Exact matching**: direct comparison of records
  - **Similarity-based methods**: clustering or distance metrics to identify near-duplicates

```
# Deduplicate based on all columns except 'order_id'
df_deduped = df.drop_duplicates(subset=df.columns.difference(['order_id']))
```

Handling Missing Values





05.dp\02-handling-missing-values

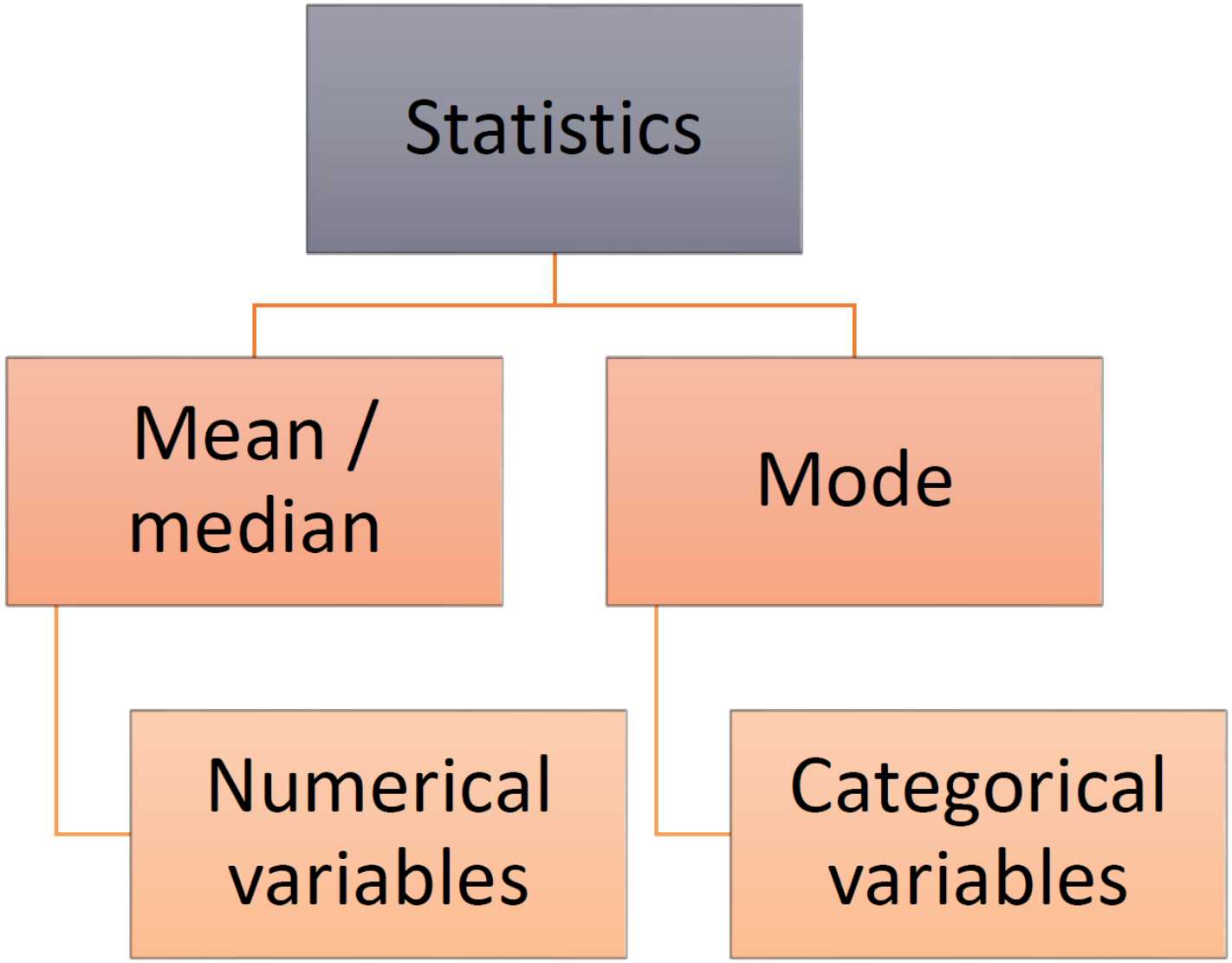
Handling missing values

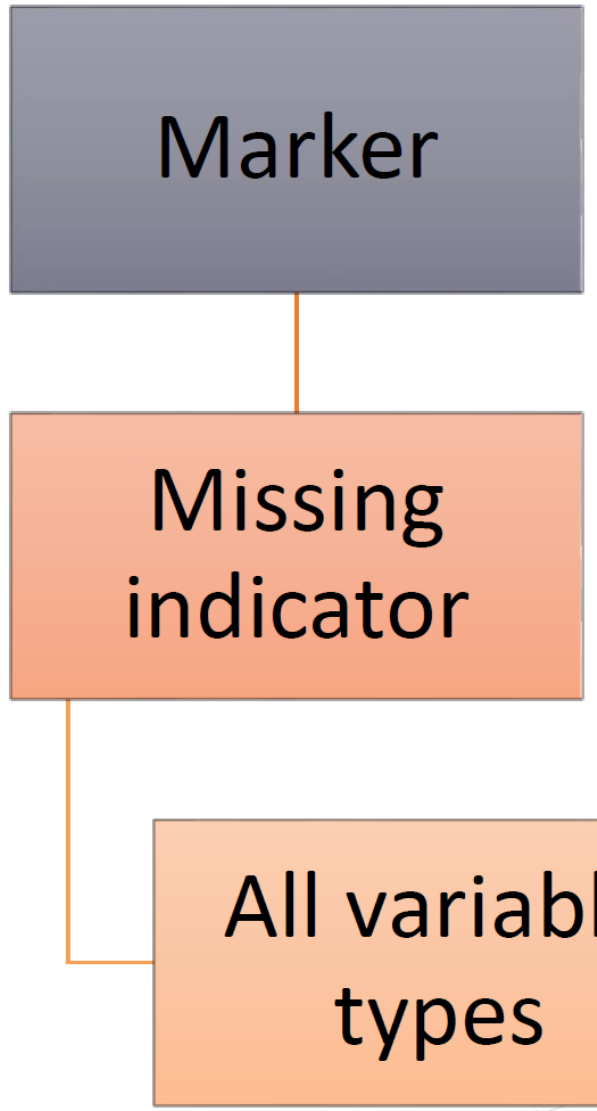
- **Remove** features or instances with excessive missingness (e.g., more than 60%)
- **Add a missing-value indicator** (flag) to explicitly mark missing data
- **Impute values** using simple statistics: Mean or median for numerical features & Mode for categorical features
- **Model-based imputation**: train an ML model to predict missing values from other features

Column 0	age	years_seniority	income	parking_space	attending_party	entree	pets	emergency_contact
Tony	48	27		1	5	shrimp		Pepper
Donald	67	25	86	10	2	beef		Jane
Henry	69	21	95	6	1	chicken	62	Janet
Janet	62	21	110	3	1	beef		Henry
Nick		17		4				
Bruce	37	14	63		1	veggie		NA
Steve	83		77	7	1	chicken		n/a
Clint	27	9	118	9		shrimp	3	None
Wanda	19	7	52	2	2	shrimp		empty
Natasha	26	4	162	5	3			_
Carol		3	127	11	1	veggie	1	""""
Mandy	44	2	68	8	1	chicken		null

https://gallery.azure.ai/Experiment/Methods-for-handling-missing-values-1

Handling missing values





Mean / Median imputation

- For numerical features, use **Mean** or **Median** imputation
  - If the data is **approximately normally distributed**, mean and median are similar
  - If the data is **skewed**, the **median** is preferred
- For categorical features, use **mode** imputation
- Assumes data is **missing at random (MAR)**
  - Missingness **depends on other features**, not on the missing value (e.g., income missing more often for younger people)

Price
100
90
50
40
20
100
60
120
200

Mean = 86.66

Median = 90



Price
100
90
50
40
20
100
86.66
60
120
86.66
200

Limitations of Mean / Median Imputation

- Distortion of the original variable distribution
  - Distortion of the original variance
  - Distortion of the covariance with the remaining features of the dataset
- The higher the percentage of missing values, the higher the distortions
- Hence limit its usage to when:
  - Data is missing completely at random
  - No more than 5% missing data per variable

Column 0	age	years_seniority	income	parking_space	attending_party	entree	pets	emergency_contact
Tony	48	27	250	1	5	shrimp		Pepper
Donald	67	25	86	10		beef		Jane
Henry	69	21	95	6				
Janet	62	21	110	3				
Nick	46	17	250	4				
Bruce	37	14	63					
Steve	83	12	77	7				
Clint	27	9	118	9		shrimp	3	None
Wanda	19	7	52	2	2	shrimp		empty
Natasha	26	4	162	5	3			_
Carol	46	3	127	11	1	veggie	1	""""
Mandy	44	2	68	8	1	chicken		null

Values are **Missing Not at Random (MNAR)** but missing because those with very high incomes preferred not to state them.
In this case, we can make a reasonable guess for what "high" means and fill in the blanks

Column 0	age	years_seniority	income	parking_space	attending_party	entree	pets	emergency_contact
Tony	48	27	250	1	5	shrimp		Pepper
Donald	67	25	86	10	2	beef		Jane
Henry	69	21	95	6	1	chicken	62	Janet
Janet	62	21	110	3	1	beef		Henry
Nick	46	17	250	4				
Bruce	37	14	63	12				
Steve	83	12	77	7				
Clint	27	9	118	9				
Wanda	19	7	52	2				
Natasha	26	4	162	5	3			_
Carol	46	3	127	11	1	veggie	1	""""
Mandy	44	2	68	8	1	chicken		null

Missing Rank Order: replace missing values with a missing rank. Our knowledge of how parking spaces are numbered let us make a guess here.

Column 0	age	years_seniority	income	parking_space	attending_party	entree	pets	emergency_contact
Tony	48	27		1	5	shrimp		Pepper
Donald	67	25	86	10	2	beef		Jane
Henry	69	21	95	6	1	chicken	62	Janet
Janet	62	21						Henry
Nick	46	17						
Bruce	37	14						NA
Steve	83	12						n/a
Clint	27	9					3	None
Wanda	19	7	52	2	2	shrimp		empty
Natasha	26	4	162	5	3			_
Carol	46	3	127	11	1	veggie	1	""""
Mandy	44	2	68	8	1	chicken		null

Interpolate: replace missing value with the average of the previous and next values

e.g.,

```
df['years_seniority'].interpolate(method='linear'))
```

Column 0	age	years_seniority	income	parking_space	attending_party	entree	pets	emergency_contact	parking_space_IsMissing
Tony	48	27	250	1					false
Donald	67	25	86	10					false
Henry	69	21	95	6					false
Janet	62	21	110	3					false
Nick	46	17	250	4					false
Bruce	37	14	63	-99	1	veggie		NA	true
Steve	83	12	77	7	1	chicken		n/a	false
Clint	27	9	118	9		shrimp	3	None	false
Wanda	19	7	52	2	2	shrimp		empty	false
Natasha	26	4	162	5	3			_	false
Carol	46	3	127	11	1	veggie	1	""""	false
Mandy	44	2	68	8	1	chicken		null	false

Replace missing values with a **Dummy Value** and create a **Missing Indicator variable** to flag missing data

Column 0	age	years_seniority	income	parking_space	attending_party	entree	pets	emergency_contact	parking_space_IsMissing	attending_party_IsMissing
Tony	48	27	250	1	5	shrimp		Pepper	false	false
Donald	67	25	86	10	2	beef		Jane	false	false
Henry	69	21	95	6	1	chicken	62	Janet	false	false
Janet	62	21	110	3	1	beef		Henry	false	false
Nick	46	17	250	4	0					true
Bruce	37	14	63	-99	1					false
Steve	83	12	77	7	1					false
Clint	27	9	118	9	0					true
Wanda	19	7	52	2	2	shrimp		empty	false	false
Natasha	26	4	162	5	3			_	false	false
Carol	46	3	127	11	1	veggie	1	""""	false	false
Mandy	44	2	68	8	1	chicken		null	false	false

Replace missing values with **0** and create a **Missing Indicator variable**

Column 0	age	years_seniority	income	parking_space	attending_party	entree	pets	emergency_contact	parking_space_IsMissing	attending_party_IsMissing	emergency_contact (2)
Tony	48	27	250	1	5	shrimp		Pepper	false	false	present
Donald	67	25	86	10	2	beef		Jane	false	false	present
Henry	69	21	95	6	1	chicken	62	Janet	false	false	present
Janet	62	21						Henry	false	false	present
Nick	46	17						no	false	true	absent
Bruce	37	14						NA	true	false	absent
Steve	83	12						n/a	false	false	absent
Clint	27	9						None	false	true	absent
Wanda	19	7						empty	false	false	absent
Natasha	26	4						_	false	false	absent
Carol	46	3	127	11	1	veggie	1	""""	false	false	absent
Mandy	44	2	68	8	1	chicken		null	false	false	absent

multiple representations for "missing"

For categorical data, missing values can be **replaced with a string** communicating that they are missing

Column 0	age	years_seniority	income	parking_space	attending_party	entree	pets	emergency_contact	parking_space_IsMissing	attending_party_IsMissing
Tony	48	27	250	1	5	shrimp		Pepper	false	false
Donald	67	25	86	10	2	beef		Jane	false	false
Henry	69	21	95	6	1	chicken	62	Janet	false	false
Janet	62	21				beef		Henry	false	false
Nick	46	17						r		
Bruce	37	14				veggie				
Steve	83	12				chicken		r		
Clint	27	9	118	9	0	shrimp	3	N		
Wanda	19	7	52	2	2	shrimp		e		
Natasha	26	4	162	5	3			_		
Carol	46	3	127	11	1	veggie	1	""""	false	false
Mandy	44	2	68	8	1	chicken		null	false	false

drop mostly empty columns

Drop mostly empty columns that are missing too many values to be useful.

drop rows missing critical values

Column 0	age	years_seniority	income	parking_space	attending_party	entree	emergency_contact	parking_space_IsMissing	attending_party_IsMissing	emergency_con (2)
					5	shrimp	Pepper	false	false	present
					2	beef	Jane	false	false	present
					1	chicken	Janet	false	false	present
Janet	62	21	110	3	1	beef	Henry	false	false	present
Nick	46	17	250	4	0		no	false	true	absent
Bruce	37	14	63	-99	1	veggie	NA	true	false	absent
Steve	83	12	77	7	1	chicken	n/a	false	false	absent
Clint	27	9	118	9	0	shrimp	None	false	true	absent
Wanda	19	7	52	2	2	shrimp	empty	false	false	absent
Natasha	26	4	162	5	3		_	false	false	absent
Carol	46	3	127	11	1	veggie	""""	false	false	absent
Mandy	44	2	68	8	1	chicken	null	false	false	absent

Remove features/instances

Remove features/instances that are excessively missing their values

```
# Creating the dataframe
df = pd.DataFrame({"A":[12, None, 1, None, 1],
                   "B":[None, 2, 2, 3, 2],
                   "C":[10, 3, 6,5, 6],
                   "D":[14, None,6, None, 6],
                   "E":[20, None,8, 3, 8],
                   "F":[10, 3, 6,5, 6],
                  })

# Print the dataframe
df
```

	A	B	C	D	E	F
0	12.0	NaN	10	14.0	20.0	10
1	NaN	2.0	3	NaN	NaN	3
2	1.0	2.0	6	6.0	8.0	6
3	NaN	3.0	5	NaN	3.0	5
4	1.0	2.0	6	6.0	8.0	6

```
#remove rows with any NaNs
dfCopy = df.copy()
dfCopy = dfCopy.dropna()
dfCopy
```

	A	B	C	D	E	F
2	1.0	2.0	6	6.0	8.0	6
4	1.0	2.0	6	6.0	8.0	6

```
#remove columns with any NaNs
dfCopy = df.copy()
dfCopy = dfCopy.dropna(axis=1)
dfCopy
```

	C	F
0	10	10
1	3	3
2	6	6
3	5	5
4	6	6

Impute with mean, median or mode

Impute with mean, median or mode of features

```
# Creating the dataframe
df = pd.DataFrame({"A":[12, None, 1, None, 1],
                   "B":[None, 2, 2, 3, 2],
                   "C":[10, 3, 6,5, 6],
                   "D":[14, None,6, None, 6],
                   "E":[20, None,8, 3, 8],
                   "F":[10, 3, 6,5, 6],
                  })

# Print the dataframe
df
```

	A	B	C	D	E	F
0	12.0	NaN	10	14.0	20.0	10
1	NaN	2.0	3	NaN	NaN	3
2	1.0	2.0	6	6.0	8.0	6
3	NaN	3.0	5	NaN	3.0	5
4	1.0	2.0	6	6.0	8.0	6

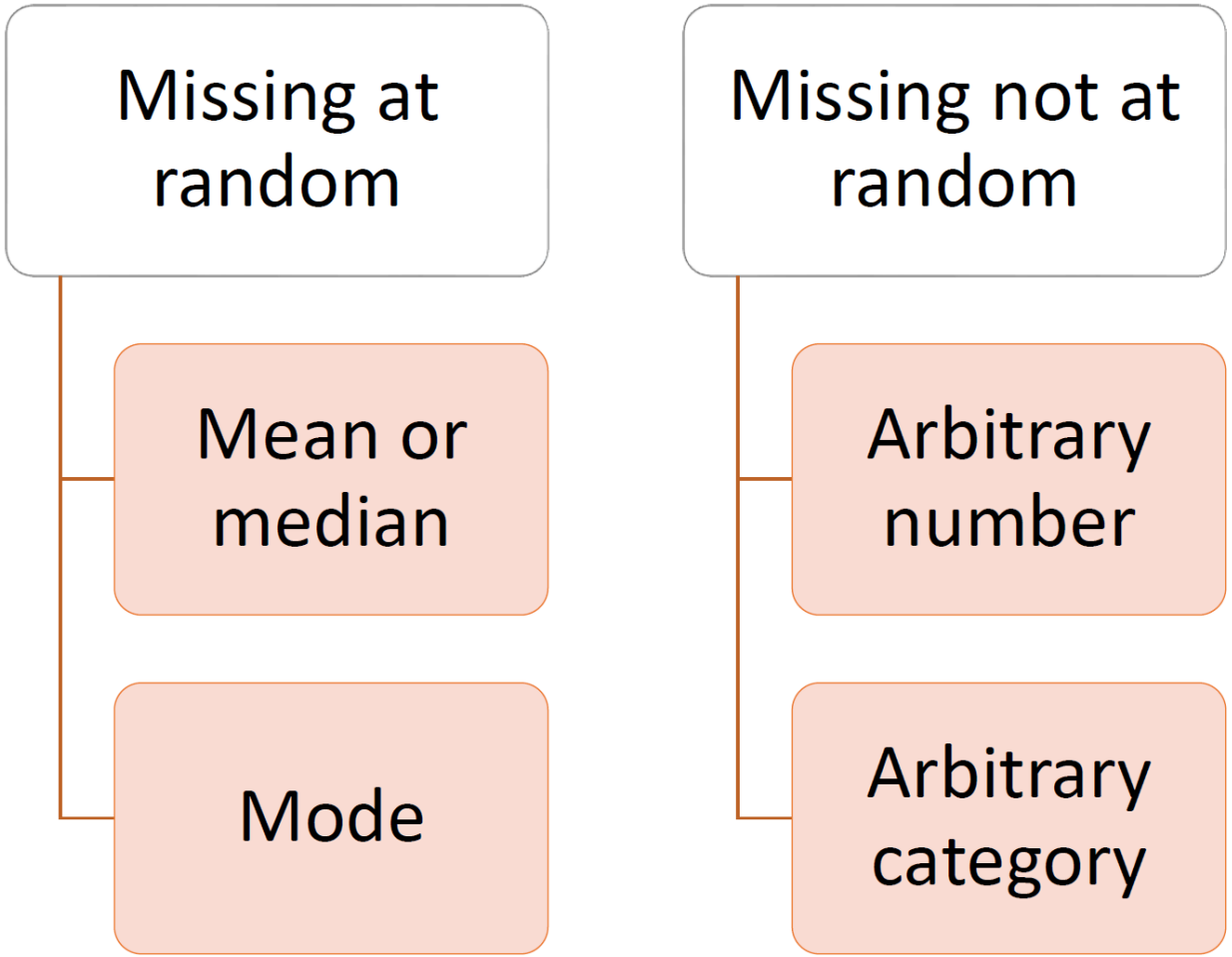
```
df.mean()  #It i
```

```
A    4.666667
B    2.250000
C    6.000000
D    8.666667
E    9.750000
F    6.000000
dtype: float64
```

```
dfCopy = df.copy()
dfCopy['A'].fillna(df['A'].mean(),inplace=True)
dfCopy['B'].fillna(df['B'].mean(),inplace=True)
dfCopy['C'].fillna(df['C'].mean(),inplace=True)
dfCopy['D'].fillna(df['D'].mean(),inplace=True)
dfCopy['E'].fillna(df['E'].mean(),inplace=True)
dfCopy
```

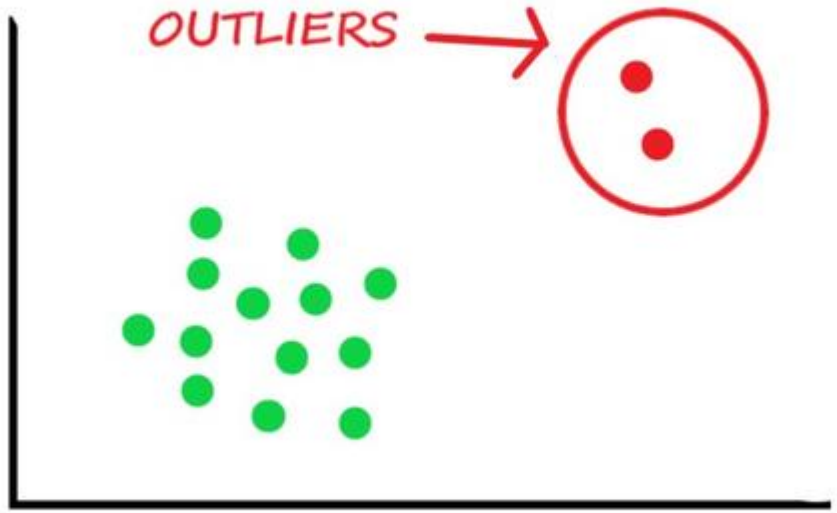
	A	B	C	D	E	F
0	12.000000	2.25	10	14.000000	20.00	10
1	4.666667	2.00	3	8.666667	9.75	3
2	1.000000	2.00	6	6.000000	8.00	6
3	4.666667	3.00	5	8.666667	3.00	5
4	1.000000	2.00	6	6.000000	8.00	6

Summary



Adding a **Missing Indicator** variable to flag missing data is a good imputation strategy

Handling Outliers



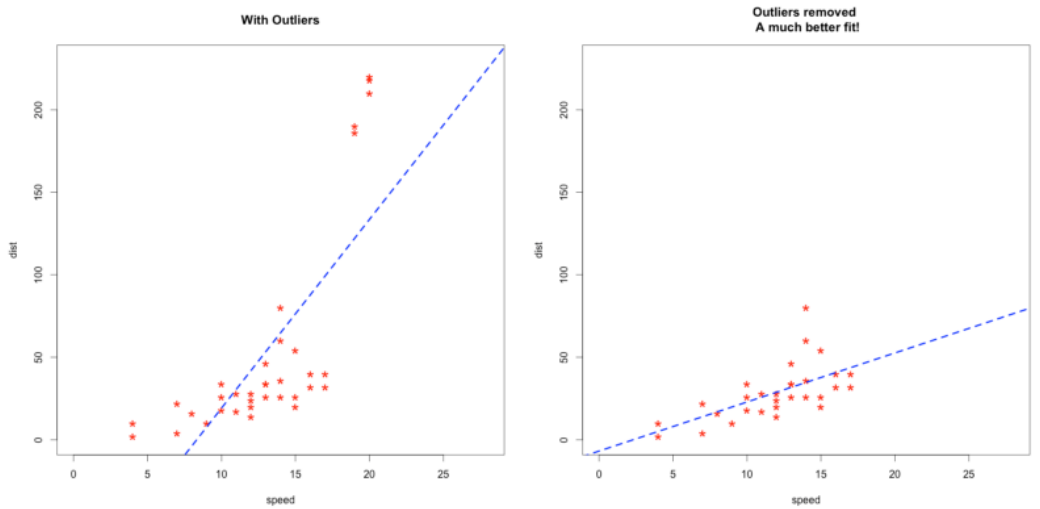


05.dp\3-handling-outliers

Handling Outliers

- Some ML techniques, e.g., logistic regression, are sensitive to outliers
- How to **handle outliers** is application-dependent, since they may:
  - Represent meaningful rare cases
  - Be noise caused by measurement or data entry errors
- In large datasets, a small number of outliers is expected and often not problematic
- Outlier detection methods:
  - **Visual**: boxplots, scatter plots, histograms
  - **Statistical**: IQR, mean ± standard deviation
  - **Algorithmic**: clustering-based approaches

https://www.r-bloggers.com/outlier-detection-and-treatment-with-r/



Ways to handle outliers

- **Trimming (Removal):** Remove outliers from the dataset
  - ✅ Fast and simple but ⚠️ may remove big chunk of data and lose information
- **Capping** (Winsorization): Replace extreme values with predefined threshold values instead of removing them (see next slide)
- **Treat outliers as missing values** and perform missing data imputation
  - ✅ No data is removed ⚠️ May distort the original distribution
- **Discretization**: Group values into bins. Place extreme values into lower or upper bins
  - ✅ Reduces outlier impact & simplifies feature representation

Ways to handle outliers - Capping

- **Capping (Winsorization):** Replace extreme values with predefined threshold values instead of removing them
  - **For skewed data (e.g., income, house prices) use IQR-based method:**

  Lower threshold = **Q1 – 1.5 × IQR**

  Upper threshold = **Q3 + 1.5 × IQR**
  - **For approximately normal data(e.g., person's height) use z-score (mean ± 3σ):**

    Lower threshold = **μ – 3σ**

    Upper threshold = **μ + 3σ**
  - Values below the lower threshold are set to the lower threshold
  - Values above the upper threshold are set to the upper threshold
  - ✅ No data is removed
  - ⚠️ May distort the original distribution

Feature Scaling

Standardisation	Normalisation
$x_{\text{stand}} = \frac{x - \text{mean}(x)}{\text{standard deviation } (x)}$	$x_{\text{norm}} = \frac{x - \min(x)}{\max(x) - \min(x)}$



05.dp\04-feature-scaling

Feature Magnitude matters

- Continuous features that cover very different ranges (i.e., Magnitude) can cause difficulty for some ML algorithms that are sensitive to feature magnitude
  - e.g., Ages range [16, 96], whereas Salaries range [10,000, 100,000]—features with large values (scales) dominate

Person	Age	Salary
1	20	25,000
2	65	25,500
3	25	25,700
4	22	26,900
5	55	25,400

- Person 1 is closer to Person 3 than Person 2, however, some distance functions will say that Person 1 and Person 2 are more similar
- Example: $\boldsymbol{distance(x, y) = \sum_{i=1}^{n} |x_i - y_i|}$

  |x_Age - y_Age| + |x_Salary - y_Salary|

  - distance(1, 2) = 45 + 500 = 545
  - distance(1, 3) = 5 + 700 = **705**

Feature scaling

- Feature scaling is the process of transforming feature values measured on different scales to **a common scale**
  - It is typically the final step in the data preprocessing pipeline, applied just before training the ML model
  - Feature scaling does not change the shape of the data distribution
- Why Feature Scaling is important:
  - Prevents features with large numeric ranges from dominating the model
  - Makes features comparable
  - Improves performance of distance-based algorithms (e.g., KNN, k-means, PCA)
  - Speeds up convergence of gradient-based optimization (e.g., neural networks, Support Vector Machine (SVMs))

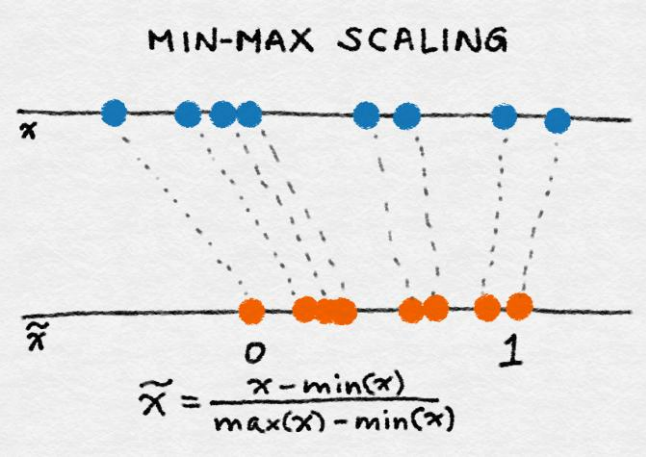
Min-Max Scaling

- **Scales the variable between 0 and 1**

$$x_i' = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

x_i' is the scaled value

- Aka **Normalization**
- Sensitive to outliers



Price
100
90
50
40
20
100
50
60
120
40
200

Max = 200
Min = 20
Range = 200 – 20 = 180

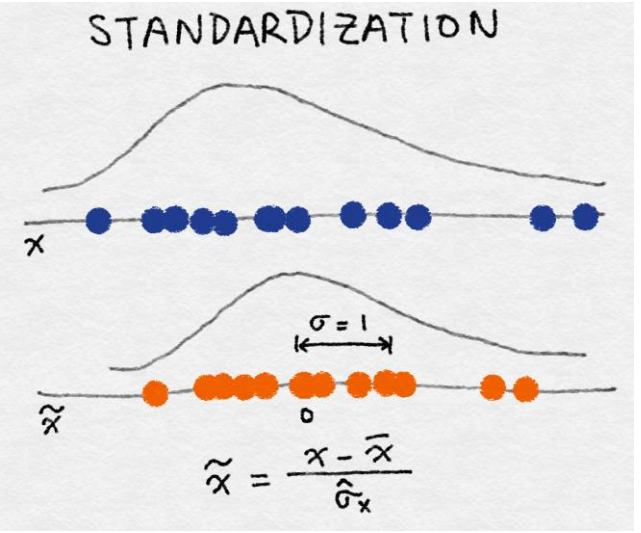
Obs. - Min

Range

Price
0.44
0.39
0.17
0.11
0.00
0.44
0.17
0.22
0.56
0.11
1.00

Image source

Standardization



- Standardization centers a feature at **zero mean** and scales it to **unit variance**

$$x_i' = \frac{x_i - \mu}{\sigma}$$

x_i' is the standardized feature value, x_i = original value, μ = mean for feature x, and σ = standard deviation

  - The standardized value indicates how many standard deviations a data point is from the mean
  - Transforms data to have mean = 0 and standard deviation = 1
  - Preserves outliers
  - Works best when the data is approximately normally distributed

Price
100
90
50
40
20
100
50
60
120
40
200

Mean = 79
Standard dev = 51

Obs. - Mean

Standard dev

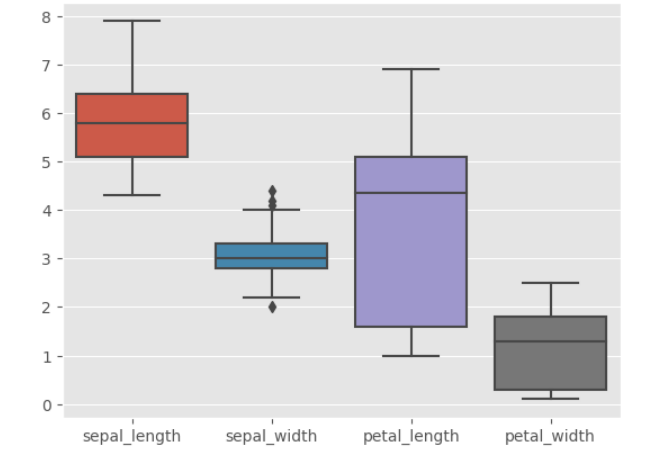
Price
0.41
0.22
-0.57
-0.76
-1.16
0.41
-0.57
-0.37
0.80
-0.76
2.37

Image source

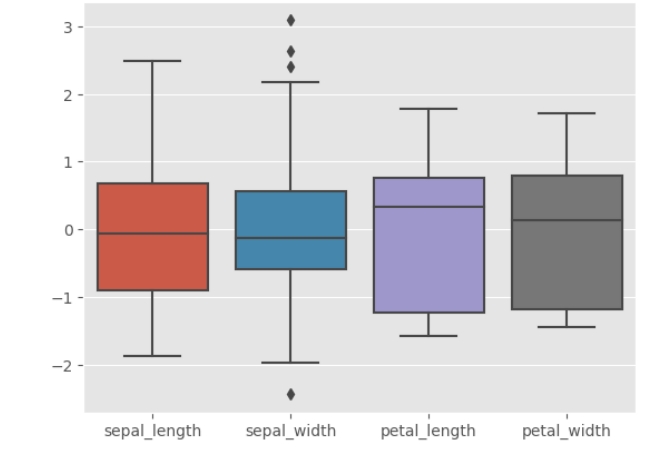
Feature scaling does not change the shape of the distribution

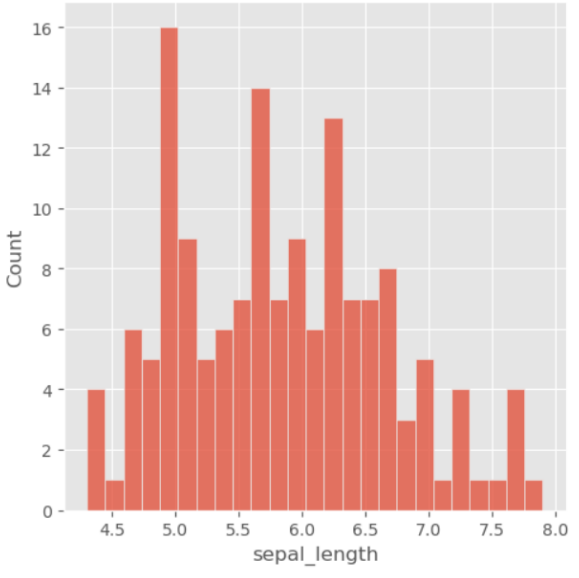
```
iris_scaled = iris.copy()
X = iris_scaled.values[:,0:-1]  #columns, except the class label, of the matrix of the dataframe
X = X.astype(float) #convert X's dtype from object to float
X_scaled = preprocessing.StandardScaler().fit_transform(X)
iris_scaled.iloc[:,0:-1] = X_scaled
iris_scaled.head()
```

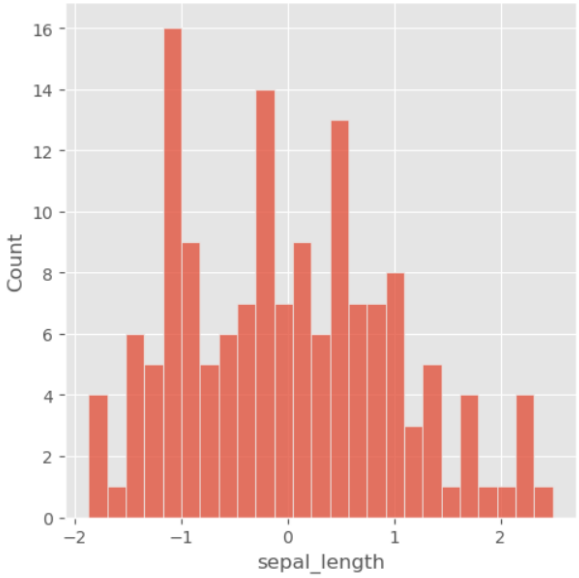
```
sns.boxplot(data=iris)
plt.show()
```




```
sns.boxplot(data=iris_scaled)
plt.show()
```

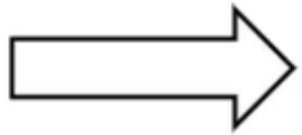






Categorical Variable Encoding

Grades
A
B
C
D
Fail



Grades	Encoded
A	4
B	3
C	2
D	1
Fail	0



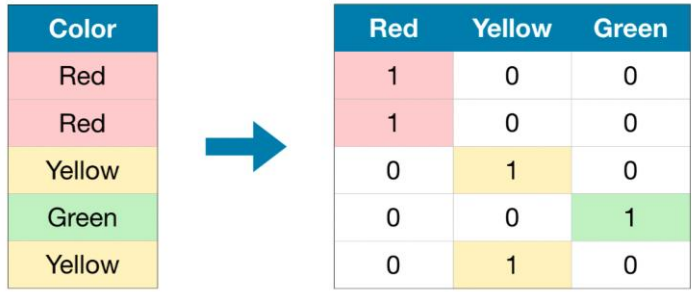
05.dp**05-categorical-encoding**

Categorical encoding

- Categorical encoding = Transform categorical (string) features into numeric representations so they can be used by ML models
- Common Encoding Methods:
  - **One hot** encoding: Creates a binary column for each category (used for Linear models)
  - **Ordinal / Label** encoding: Assigns an integer to each category (used for Tree based models)
  - **Count / frequency** encoding: Replaces each category with its frequency (used when there are many categories)

One hot encoding

- Converts a categorical feature into multiple **binary (0/1) indicator variables**, one per category
  - A value of **1** indicates the presence of the category, and **0** indicates its absence



Color
Red
Red
Yellow
Green
Yellow

Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1
0	1	0

- **When to Use:** Suitable for **linear models** and algorithms that assume no ordinal relationship between categories
- **Limitations**
  - **Large feature expansion** when a variable has many categories (high cardinality)
  - Can introduce **redundant or identical dummy variables**, adding no new information

Ordinal Encoding

- Assigns **each category a unique integer** value (e.g., 0 to n-1 or 1 to n, where n is the number of distinct categories)
  - 👍 Does not expand the feature space
- **When to Use**:
  - Works well with tree-based models, which can handle arbitrary integer labels
- **Limitations:**
  - Not suitable for linear models or distance-based algorithms, since they may incorrectly interpret the numeric labels as having meaningful order or magnitude

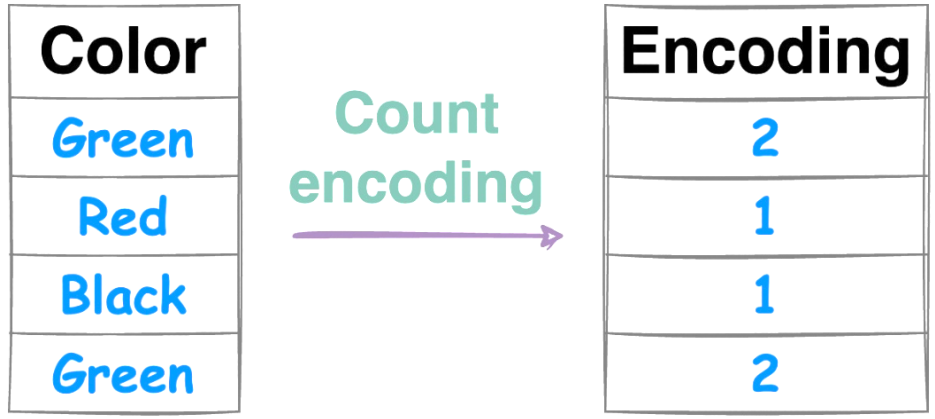
Color
Green
Red
Blue



Color
1
2
3

Count / frequency encoding

- Replaces each category with its **count** or **frequency** in the dataset
  - Does not expand the feature space



When to Use

- Useful for **high-cardinality** categorical variables
- Works well with **tree-based models**

Limitations

- **Not suitable for linear models**
- Categories with the **same frequency get the same value**
  - Can lead to **loss of information**